STANDALONE PRODUCT DIRECTIONS

Mobile Event-Day Alerts

Build the accounts, event data, push service, and scheduler inside this project

Standalone app | Email sign-in only | Mobile web push | No PIXL Sports connection

Instruction for Replit

Build a self-contained event app with its own authentication, persistent database, API, mobile web-push subscriptions, reminder preferences, and scheduled delivery. Do not connect to, import from, or depend on the PIXL Sports app, API, database, Clerk tenant, or user accounts.

What the completed system does

Account
Person creates an account using email sign-up/sign-in only.

Event
Owner saves an event with its local date, time, and time zone.

Phone
Owner grants permission and opts this device into push.

Scheduler
Timed job wakes the app's backend and claims due reminders.

Alert
Backend sends; service worker displays and opens the app.

Mobile web push

Use a responsive website / installable web app with HTTPS, a service worker, and browser Web Push. This is not a native iOS or Android app.

Explicit choice

Installing a shortcut or granting browser permission does not, by itself, opt a user into event reminders. Ask for each choice clearly in the app.

Keep the scope independent

Use this project's own user IDs and persistent data. If Clerk is selected for sign-in, configure a separate Clerk app/tenant for this project and enable email only. Do not expect existing PIXL accounts to work here.

CREATE EACH LAYER IN THIS REPLIT PROJECT

Own the event and alert records

Authentication

Set up this project's sign-up/sign-in and stable internal user ID. Offer email only for the first release; do not add Google. Every private event, preference, subscription, and delivery record must be scoped to that ID.

Persistent database

Use a database owned by this project. Do not keep the event schedule, alert opt-in, or delivery history only in browser storage; those must survive refreshes and work when the phone is offline.

Event records

Store an event ID, owner ID, type (game/practice), local date, optional local start time, IANA time zone, and status. Changes, cancellation, and finalization must update or cancel pending reminders.

Alert records

Store account reminder choices and alert time zone separately from device subscriptions. Keep durable delivery rows or an equivalent ledger so scheduler retries cannot create duplicates.

Suggested API shape for this new app

Method + path	Purpose	Access
GET `/api/notifications/config`	Return browser support / push availability and the VAPID public key	Public
POST `/api/notifications/subscriptions`	Save the signed-in user's device endpoint, browser keys, and zone	Signed in
DELETE `/api/notifications/subscriptions`	Remove this user's device subscription	Signed in
GET / PATCH `/api/notifications/preferences`	Read / save one-hour, day-of, and alert-zone choices	Signed in
POST `/api/internal/jobs/event-alerts/cron`	Protected scheduled worker entry point	Scheduler only

Data model guardrail

Every private read/write checks the authenticated user ID on the server. A public event page, if added later, must use a separate share token and a deliberately filtered response—not expose owner records or push subscription data.

These paths are proposed for the new project. Build and document its own API contract; they are not routes on another app.

ASK ONLY AFTER THE USER TAPS ENABLE

Connect a browser subscription safely

1. **Serve the app securely**
   Deploy the app on HTTPS. Register a service worker from this app's origin and scope it to the app. Mobile browsers require a supported browser/OS; validate the specific devices you intend to support.

2. **Configure Web Push credentials**
   Create a VAPID key pair for this app. The API returns only `VAPID_PUBLIC_KEY` to the signed-in client; keep `VAPID_PRIVATE_KEY` and `VAPID_SUBJECT` in this project's server-side Replit Secrets.

3. **Check browser capabilities**
   Before showing the enable action, confirm the browser has `serviceWorker`, `PushManager`, and `Notification`. Give a plain explanation if the device or browser cannot receive web push.

4. **Request permission from a button**
   After the person taps "Enable Phone Alerts," request browser permission, wait for `serviceWorker.ready`, then call `pushManager.subscribe({ userVisibleOnly: true, applicationServerKey })` with this app's public VAPID key.

5. **Save the subscription to this app's API**
   Send the browser endpoint and `keys.p256dh` / `keys.auth` over the authenticated session. Store them against this project's user ID; do not log endpoints or keys. Keep a subscription per device/browser.

6. **Let the user manage it**
   Show whether this device is connected, offer a one-device test alert, and let the user turn the device off. Turning off removes the server record and calls the browser subscription's `unsubscribe()`.

State mobile support limits plainly
On iPhone and iPad, web push requires a Home Screen web app on iOS/iPadOS 16.4 or later; users must add the site to the Home Screen before opting in. Support also varies across browsers, embedded webviews, and restricted modes, while OS notification settings can suppress or delay display. Test and list the exact browser/OS combinations supported; do not promise universal delivery.

Keep permission and reminder consent distinct
Browser permission means the operating system may show alerts. A saved device subscription means the server can address that browser. Separate account checkboxes decide whether one-hour and/or day-of event reminders should be scheduled. Missing preferences default to off.

The service worker should show a notification on `push` and open a safe app URL on `notificationclick`. Do not cache private API responses in the worker.

RECOMMENDED FIRST-RELEASE BEHAVIOR

Use explicit preferences and local time

One hour before

For a timed event, calculate one hour before its saved local start using its IANA event time zone. Do not schedule this for date-only events.

Day of the event

Schedule one reminder at **7:00 a.m.** in the account's selected alert time zone, if that is before the event starts.

Default off

New accounts have both reminder types off until the owner explicitly chooses them. Installing the app or enabling browser permission must not change this.

Multiple devices

Send to the owner's currently active subscriptions, while recording a separate attempt/result per device. Turning off one device should not erase other devices.

Time-zone and schedule handling

- Save event-local date/time and a validated IANA time-zone name. Resolve the due instant to UTC for storage and comparison.
- Save the account alert time zone separately. Do not infer account consent or timing from the most recently used device.
- Define daylight-saving behavior: skip a nonexistent local time at the spring transition; use the first occurrence of a repeated fall-back time.
- Use a unique delivery key for each owner, event, event date, reminder type, and target device (or an equivalent design that prevents duplicates across retries).
- Re-check the latest event status, owner preference, and subscription before sending. Event edits reschedule; cancellation/finalization stops stale pending reminders.
- Set a relevance window so an outage does not cause very late alerts. A reasonable design can recover a missed one-hour reminder with a clearly labeled "event is coming up" fallback for a limited post-start window.

Example title

`GAME DAY · IN 1 HOUR`

Example title

`PRACTICE · TODAY`

Example title

`GAME DAY · COMING UP`

Keep audiences separate

These scheduled reminders go only to the signed-in event owner's opted-in devices. They are not an in-app dashboard prompt and do not send to friends, followers, or the public automatically. Any future follower announcement needs its own explicit action, recipient eligibility, and consent.

A push provider accepting a message is not proof that the phone displayed it or the person read it. Track "sent," "failed," and "uncertain" outcomes accurately; do not promise delivery during device or provider outages.

DO NOT RELY ON AN IN-PROCESS TIMER

Wake the backend on a schedule

Autoscale processes can stop when idle, so an in-memory `setInterval` is not a reliable event reminder scheduler. Keep the app/API on its intended hosting and use a recurring external caller (for example, cron-job.org) to invoke a protected HTTPS endpoint every five minutes.

1. **Keep the sender in the app backend**
   The backend owns the persistent database, subscription records, delivery ledger, and Web Push library. The scheduled caller only wakes the backend; it never receives the database URL or VAPID private key.

2. **Set server-side secrets in this Repl**
   Configure `VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`, and a random scheduler token named `EVENT_ALERTS_CRON_TOKEN`. Use Replit Secrets for production; do not put values in source, browser code, URLs, or this guide.

3. **Create a narrowly scoped job endpoint**
   Implement `POST /api/internal/jobs/event-alerts/cron`. It should authenticate a fresh timestamp, random nonce, and token; reject stale/replayed requests; claim a bounded batch; re-check current eligibility; record results; and return only aggregate counts.

4. **Configure one recurring caller**
   Use the current published HTTPS origin for this new Replit app, method `POST`, every five minutes (`*/5 * * * *`), and no body or URL parameters. Keep the schedule disabled until setup is reviewed and approved.

5. **Put credentials in headers**
   For cron-job.org, configure fresh values on every run. Header names are a proposed contract for this new project:

Header	Example value / purpose
`x-event-alerts-timestamp`	Fresh Unix timestamp supplied by the scheduler
`x-event-alerts-nonce`	Fresh unique UUID per execution
`x-event-alerts-token`	The value stored in this Repl's `EVENT_ALERTS_CRON_TOKEN` secret

Never test the production job with a POST
The request can send real phone alerts. Test the scheduler logic with fake subscriptions and a non-delivering sender in development. Before enabling production, review the public URL, production secrets, schema, time zone behavior, single-caller rule, and any displayed hosting or service costs.

Replit Scheduled deployments run cron-like programs; they are not the web/API host. If choosing that model for the worker, verify how it accesses this project's own persistent data and how it coexists with the web/API deployment. For an external caller, ensure the published endpoint is reachable; if app privacy blocks it, use Replit's supported external-access method and keep its token in an authorization header, never a URL.

BUILD CONFIDENCE WITHOUT ALERTING REAL USERS

Verify the complete path

Safe device test

On a dedicated test account and phone, enable browser permission and this device, choose a reminder, and use a one-off “Send Test Alert” control. Confirm display and click-through; then turn the test device off and remove its subscription.

Safe scheduler test

Use fixtures and a fake sender to test due-time selection, duplicates, edits, cancellation, retries, overlapping jobs, and time zones. Do not send to real production subscriptions during automated tests.

Read-only health check

Add an authenticated status check that reports last successful run and aggregate pending/sent/failed/uncertain counts. A status request must never send notifications.

Useful alerts for operators

Surface failed job calls, authentication/replay errors, growing due backlog, repeated deferrals, and prolonged lack of successful checks. Never log tokens, subscription URLs, or private event data.

Release acceptance checklist

- [ ] This Replit project has its own authentication, email-only sign-in, persistent database, event records, and API; no PIXL Sports integration or data dependency exists.
- [ ] The mobile web app is served over HTTPS; a supported browser can opt in, receive a push, and open the correct event/app screen.
- [ ] Device subscription and account reminder consent are separate; both reminder types default off and can be turned off.
- [ ] One-hour alerts use the event time zone; day-of alerts use 7:00 a.m. in the account alert time zone.
- [ ] Event changes, cancellations, and final results update pending work; retries, concurrent runs, and multiple devices do not create duplicate alerts.
- [ ] VAPID private key and scheduler token are server-side secrets; the scheduled job is authenticated, replay-safe, bounded, and monitored.
- [ ] Production scheduling is enabled only after the test plan, live endpoint, and any displayed costs are reviewed.

Recommended implementation order

Build authentication and event CRUD → persistent preferences and subscriptions → service-worker test push → time-zone calculation and delivery ledger → protected scheduled endpoint → non-delivering tests and operational status → approved production activation.

This document is implementation direction only. It does not create accounts, secrets, an external cron job, a database, a deployment, or any production notifications.